

Lab Report: CAP Theorem

Name: Robert Mesek

Lab: 5

Date: April 16, 2026

Assignment

Submit a report that explains the following:

1. (3p) Answer the questions found above in the description of Exercises 1,2,3.
2. (2p) Which data structures -- AP or CP -- require the Raft algorithm? Why is this algorithm needed? After partition heal, what is the role of Raft?
3. (2p) Read article Session Guarantees for Weakly Consistent Replicated Data and explain what it means that the PN Counter in Hazelcast provides Read-Your-Writes (RYW) and Monotonic reads guarantees and why they are session guarantees.

Solution

1. Answers for Exercises

Ex.1 Explain if and why you were/were not able to get/increase the value of **AtomicLong** in each step on particular nodes.

Answer: During the partition, Nodes 1 and 2 could still process reads and writes because they maintained a quorum (2 out of 3 nodes). Node 3 timed out on both operations because it was isolated and lost quorum. The **AtomicLong** is a CP structure using Raft, which sacrifices availability to ensure strong consistency.

Terminal output:

```
=====
Exercise 1: CP - One Node Isolated (CP guarantees)
Data structure: IAtomicLong (CP)
Nodes: 3 | Timeout: 10s
=====

Commands:
add:N / a:N      Increment counter on node N
get:N / g:N      Get counter value from node N
getAll / ga      Get counter value from all nodes
partition / p    Simulate network partition
heal             Restore network connectivity
status / s       Show cluster status
help / h         Show this help
exit / q         Quit

[OK]> getAll
Node | Value
```

```

-----+-----
1      |  0
2      |  0
3      |  0
[OK]> add:1
Node 1 += 1 -> 1
[OK]> getAll
Node  |  Value
-----+-----
1      |  1
2      |  1
3      |  1
[OK]> partition
Node 3 isolated from Nodes 1 and 2. (Majority: nodes 1,2)
[SPLIT]> get:3
Node 3 -> (timeout - no quorum)
[SPLIT]> get:1
Node 1 -> 1
[SPLIT]> get:2
Node 2 -> 1
[SPLIT]> add:3
Node 3 += 1 -> (timeout - no quorum)
[SPLIT]> add:1
Node 1 += 1 -> 2
[SPLIT]> heal
Partition healed. All nodes can communicate.
[OK]> getAll
Node  |  Value
-----+-----
1      |  2
2      |  2
3      |  2

```

Ex.2 Explain if and why you were/were not able to get/increase the value of `AtomicLong` in each step on particular nodes.

Answer: During the partition, all nodes were isolated. None could form a quorum (2 out of 3), so all read and write operations timed out. This strict behavior prevents split-brain and ensures CP guarantees. After healing, quorum was restored and operations succeeded.

Terminal output:

```

=====
Exercise 2: CP - All Nodes Isolated  (CP guarantees)
Data structure: IAtomicLong (CP)
Nodes: 3 | Timeout: 10s
=====

Commands:
add:N / a:N      Increment counter on node N
get:N / g:N      Get counter value from node N

```

```

getAll / ga      Get counter value from all nodes
partition / p    Simulate network partition
heal             Restore network connectivity
status / s      Show cluster status
help / h        Show this help
exit / q        Quit

```

```

[OK]> getAll
Node | Value
-----+-----
1    | 0
2    | 0
3    | 0
[OK]> add:1
Node 1 += 1 -> 1
[OK]> getAll
Node | Value
-----+-----
1    | 1
2    | 1
3    | 1
[OK]> partition
All nodes isolated from each other. (No quorum possible)
[SPLIT]> get:1
Node 1 -> (timeout - no quorum)
[SPLIT]> get:2
Node 2 -> (timeout - no quorum)
[SPLIT]> add:3
Node 3 += 1 -> (timeout - no quorum)
[SPLIT]> heal
Partition healed. All nodes can communicate.
[OK]> getAll
Node | Value
-----+-----
1    | (timeout - no quorum)
2    | (timeout - no quorum)
3    | (timeout - no quorum)
[OK]> getAll
Node | Value
-----+-----
1    | 1
2    | 1
3    | 1

```

Ex.3 Explain if and why you were/were not able to get/increase the value of PNCounter in each step on particular nodes.

Answer: During the partition, all reads and writes succeeded across both connected and isolated nodes. **PNCounter** is an AP data structure (CRDT) which prioritizes availability and does not require a quorum. After healing, the divergent concurrent increments inherently merged into a globally consistent state safely (eventual consistency).

Terminal output:

```
=====
Exercise 3: AP - PNCounter (CRDT) (AP guarantees)
Data structure: PNCounter (AP)
Nodes: 3 | Timeout: 10s
=====
```

Commands:

```
add:N / a:N      Increment counter on node N
get:N / g:N      Get counter value from node N
getAll / ga      Get counter value from all nodes
partition / p    Simulate network partition
heal             Restore network connectivity
status / s      Show cluster status
help / h        Show this help
exit / q        Quit
```

```
[OK]> getAll
```

Node	Value
1	0
2	0
3	0

```
[OK]> add:1
```

```
Node 1 += 1 -> 1
```

```
[OK]> getAll
```

Node	Value
1	1
2	1
3	1

```
[OK]> partition
```

```
Node 3 isolated from Nodes 1 and 2.
```

```
[SPLIT]> get:3
```

```
Node 3 -> 1
```

```
[SPLIT]> get:1
```

```
Node 1 -> 1
```

```
[SPLIT]> get:2
```

```
Node 2 -> 1
```

```
[SPLIT]> add:3
```

```
Node 3 += 1 -> 2
```

```
[SPLIT]> add:1
```

```
Node 1 += 1 -> 2
```

```
[SPLIT]> getAll
```

Node	Value
1	2
2	1
3	2

```
[SPLIT]> heal
```

```
Partition healed. All nodes can communicate.
```

```
[OK]> getAll
```

Node		Value
-----+-----		
1		3
2		3
3		3

2. Raft algorithm in AP and CP data structures

CP data structures require the Raft algorithm. It is needed to achieve distributed consensus, ensuring a strict majority (quorum) agrees on state changes, which prevents split-brain scenarios and ensures strong consistency. After a partition heals, Raft's role is to elect a stable leader and synchronize the internal logs of the rejoining nodes so their state catches up with the majority.

3. Session guarantees for PN Counter in Hazelcast

- **Read-Your-Writes (RYW):** A client will always see the effects of its own completed writes within the same session.
- **Monotonic reads:** Successive reads by a client within the same session will never return older data than what it read previously.
- **Why they are session guarantees:** They are scoped to the context of a specific client's session rather than being strict global guarantees across all clients. This allows the cluster to maintain high availability (AP) and eventual consistency globally, while still giving an individual client a predictable and coherent view of their own operations.